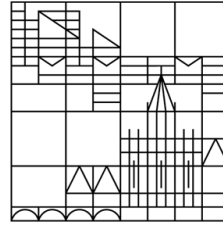


Universität  
Konstanz



# Detailed Documentation

*Homing with a Bayesian Ring Attractor*

---

Documentation and results of the  
Bayesian ring-attractor-based homing system

---

**Louis Renard**

University of Konstanz

Supervised by: A. Reina, S. Atasoy Bingöl, S. Mkhathshwa  
In collaboration with: V. Radoynovski, A. Doimo

August 20, 2026

# Contents

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>introduction</b>                                          | <b>2</b>  |
| 1.1      | Motivations . . . . .                                        | 2         |
| <b>2</b> | <b>System Overview</b>                                       | <b>2</b>  |
| <b>3</b> | <b>Hardware</b>                                              | <b>4</b>  |
| 3.1      | Raspberry Pi . . . . .                                       | 4         |
| 3.2      | Drive System and Motor Controller . . . . .                  | 4         |
| 3.3      | Inertial Measurement Unit . . . . .                          | 5         |
| 3.4      | 360 Camera . . . . .                                         | 6         |
| 3.5      | Microphone Array . . . . .                                   | 7         |
| <b>4</b> | <b>Software Architecture</b>                                 | <b>7</b>  |
| 4.1      | Main Realtime Pipeline . . . . .                             | 8         |
| 4.2      | Data Acquisition and Concurrency . . . . .                   | 8         |
| 4.3      | Dynamic Sensor Weighting . . . . .                           | 9         |
| 4.4      | Robot Control Interface . . . . .                            | 11        |
| 4.5      | Logging and Realtime Visualization . . . . .                 | 11        |
| <b>5</b> | <b>Bayesian Ring attractor</b>                               | <b>12</b> |
| 5.1      | Initialisation and Network Parameters . . . . .              | 12        |
| 5.1.1    | Weight Matrix Construction . . . . .                         | 12        |
| 5.1.2    | External Input . . . . .                                     | 13        |
| 5.2      | Network Dynamics: <code>step()</code> . . . . .              | 13        |
| 5.3      | Decoding $\mu_t$ and $\kappa_t$ . . . . .                    | 14        |
| 5.4      | Parameters and Their Effects . . . . .                       | 14        |
| 5.4.1    | $N$ : Number of Neurons . . . . .                            | 14        |
| 5.4.2    | $\tau$ : Membrane Time Constant . . . . .                    | 15        |
| 5.4.3    | $\kappa_\phi$ : Inverse Diffusion Constant . . . . .         | 15        |
| 5.4.4    | $\kappa_{vt}$ : Total Angular Velocity Reliability . . . . . | 15        |
| 5.4.5    | $\kappa_z$ : HD Observation Reliability . . . . .            | 15        |
| 5.4.6    | $w_{quad}$ : Quadratic Inhibition Strength . . . . .         | 15        |
| 5.5      | Output . . . . .                                             | 15        |
| <b>6</b> | <b>Results</b>                                               | <b>15</b> |
| 6.1      | Model performance score . . . . .                            | 15        |
| 6.2      | h optimization . . . . .                                     | 16        |
| 6.3      | Effect of stationary vs motion phases . . . . .              | 17        |
| 6.4      | Adaptive weights difference . . . . .                        | 18        |
| 6.5      | Ground truth comparison . . . . .                            | 18        |
| <b>7</b> | <b>Conclusion</b>                                            | <b>19</b> |

# 1 introduction

This project stems from the research of Kutschireiter et al. [2], who demonstrated how a ring attractor network can implement Bayesian inference over a circular variable while simultaneously representing the uncertainty of its estimate. Building upon this framework, this project investigates its application to robotic heading estimation and homing. This project uses acoustic sources as targets which serve as inputs to the model along with onboard odometry sensors, and particularly focuses on how to fuse these inputs together.

The main real-time pipeline is `SoundBayesianRing.py` which collects readings from a 360 camera, a gyroscope and an IMU magnetometer, and fuses the sensor readings together. It then estimates direction of arrival of the incoming sound and its associated uncertainty before updating the bayesian ring attractor model for one time step given these inputs. This allows the model to keep a real-time belief of its heading direction relative to home.

The experiment consisted of fixing a speaker playing a constant sound in a motion capture arena. The robot then drives around the arena, collecting sensor readings, and running the Bayesian ring pipeline in real time. The motion capture is used for ground truth measurements.

## 1.1 Motivations

Ring attractors have been observed in nature to serve as the basis for heading estimation and path integration. As for the Bayesian Ring attractor, the concept of uncertainty baked into the model itself has proven relevant in research led by Basnak et al. [1] which showed that fruit flies possessed the capabilities to integrate cues of varying informativeness. This leads us to believe that the bayesian ring attractor dynamics perhaps resemble more closely that of animals. Implementing this model on a robot also opens the door to the possibility of implementing a similar system on a neuromorphic circuit for an even greater biological plausibility.

The decision to use a sound source as a home target was due to the high uncertainty present when working with sound, especially in rooms not designed to minimize echo and interference. This means that a model that is able to distinguish between "good" and "bad" cues was hypothesized to perform well. With this projected we were curious to discover if the bayesian ring still performed well when transferred to hardware and what can be learned in the process.

## 2 System Overview

The system implements a bio-inspired approach to robotic heading estimation and homing. The robot estimates its heading relative to a target while moving through its environment by combining multiple sensory cues with different levels of uncertainty. These cues are processed independently and subsequently fused by the Bayesian ring attractor to produce a circular probability distribution over the robot's heading.

The overall architecture of the system is shown in Figure 1. Three primary sensory modalities

are used. The BNO055 inertial measurement unit (IMU) provides measurements of angular velocity and orientation, the omnidirectional camera provides rotational information through visual odometry, and the ReSpeaker microphone array provides an acoustic direction-of-arrival (DOA) estimate of the target sound. Each modality is processed independently before being provided to the Bayesian ring attractor.

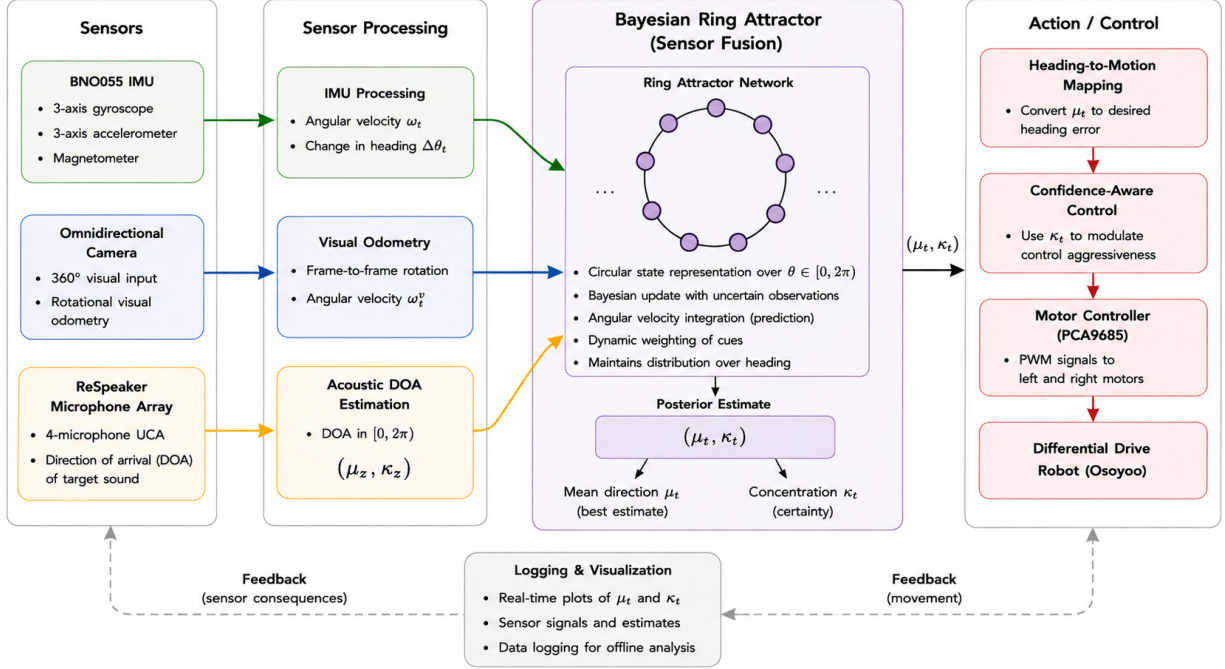


Figure 1: Overview of the robotic Bayesian ring-attractor system.

The Bayesian ring attractor represents heading as a circular variable,

$$\theta \in [0, 2\pi), \quad (1)$$

and maintains a probabilistic estimate of the current state. Rather than representing only a single heading value, the model represents both the estimated direction and the uncertainty associated with that estimate. The circular mean  $\mu$  represents the most likely heading, while the concentration parameter  $\kappa$  represents the certainty of the estimate.

The sensory observations are therefore not treated as equally reliable measurements. Instead, each observation is associated with a corresponding concentration parameter. A highly reliable observation has a larger  $\kappa$  and consequently exerts a stronger influence on the posterior estimate, whereas an uncertain observation has a lower  $\kappa$  and contributes less strongly to the fused estimate. This allows the system to integrate information from sensors with substantially different noise characteristics.

Angular velocity measurements provide the temporal component of the estimation process. The previous state of the ring attractor is propagated according to the measured rotational motion, while external sensory observations provide information that can correct this prediction. The acoustic DOA measurement is particularly important in the homing task because it

provides a directional observation of the target, while the IMU and visual odometry provide information about how the robot itself has rotated.

The resulting posterior estimate is described by

$$(\mu_t, \kappa_t), \quad (2)$$

where  $\mu_t$  is the estimated heading at time  $t$  and  $\kappa_t$  is its associated certainty. These quantities are passed to the robot-control system. The heading estimate determines the direction in which the robot should turn, while the concentration is used to modulate the confidence of the resulting motion. The robot can therefore behave more decisively when the estimated heading is reliable and more conservatively when the available sensory information is uncertain.

The complete real-time pipeline is implemented on a Raspberry Pi. Sensor acquisition, visual processing, acoustic processing, Bayesian inference, and motor control are implemented as separate software components. The resulting architecture allows the same Bayesian estimation framework to be evaluated both independently as a state-estimation system and as part of a closed-loop physical homing experiment.

### 3 Hardware

The physical implementation of the Bayesian ring attractor was built around a small mobile OSOYOO robot controlled by a Raspberry Pi. The Raspberry Pi acts as the central processing unit, interfacing with the robot’s drive system, an inertial measurement unit (IMU), a 360 camera, and a microphone array. These sensors provide complementary measurements of the robot’s heading and environment, which are subsequently fused by the Bayesian ring attractor.

#### 3.1 Raspberry Pi

A Raspberry Pi is used as the main computational platform for the system. The realtime software is designed to run directly on the Raspberry Pi and makes use of its GPIO and I2C interfaces. The Python implementation communicates with the hardware through `RPi.GPIO`, `Adafruit_PCA9685`, and the Adafruit BNO055 libraries.

The Raspberry Pi runs the main realtime pipeline implemented in `SoundBayesianRing.py`. During operation, the program continuously acquires sensor measurements, computes the corresponding odometry and heading observations, and passes them to the Bayesian ring attractor. The estimated mean direction  $\mu$  and concentration  $\kappa$  can then be sent to the motor-control system.

#### 3.2 Drive System and Motor Controller

The robot is driven by two independently controlled motors. A PCA9685 PWM controller is used to generate the motor-control signals. The controller is configured with a PWM

frequency of 60, Hz and provides separate PWM channels for the left and right motor enable signals.

The motor direction signals are connected directly to Raspberry Pi GPIO pins. The implementation in `robot_toy.py` uses GPIO pins 23 and 24 for the first motor and GPIO pins 27 and 22 for the second motor. The PCA9685 channels 0 and 1 are used for the corresponding motor enable signals.

The software provides several basic motion primitives:

- forward motion using equal positive motor commands;
- backward motion using equal negative motor commands;
- left and right arcs using unequal motor speeds;
- in-place rotation by driving the two motors in opposite directions;
- an explicit stop command which disables both motors.

The nominal software values are a drive command of 1300 and turning and spinning commands of 1400. These values correspond to the PWM command range used by the PCA9685 interface rather than directly representing a physical velocity. These can be changed however depending on the robot driving surface or if faster movement is required.

For autonomous homing, the motor controller receives the estimated heading  $\mu$  and the concentration  $\kappa$  from the Bayesian ring attractor. The sign of  $\mu$  determines the direction in which the robot turns, while  $\kappa$  determines the duration of the resulting turn and forward movement. In the current implementation,

$$t_{\text{move}} = 0.01 + 0.02\kappa \quad (3)$$

and

$$t_{\text{turn}} = 0.01 + 0.01\kappa. \quad (4)$$

Consequently, the estimated uncertainty is incorporated directly into the robot’s behaviour: a larger  $\kappa$  results in a more decisive movement, whereas a lower  $\kappa$  produces shorter actions. The motor-control loop runs in a separate worker thread, with a queue containing only the most recent heading estimate. This prevents outdated estimates from accumulating when the perception pipeline produces observations faster than the robot can execute them.

### 3.3 Inertial Measurement Unit

A BNO055 inertial measurement unit is connected to the Raspberry Pi using the I2C interface. The sensor provides both gyroscope measurements and world heading measurements which are integrated to calculate angular velocity. The implementation accesses the BNO055 through the `board`, `busio`, and `adafruit_bno055` Python libraries.

The IMU is wired using the following configuration:

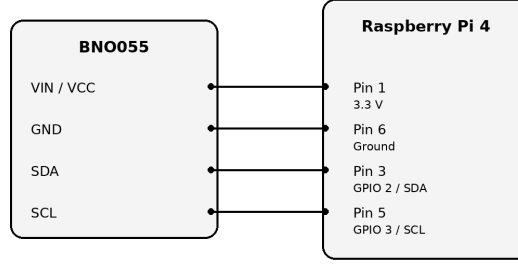


Figure 2: BNO055 I2C connection to raspberry pi 4

The IMU is sampled continuously in a dedicated thread. The software reads the gyroscope and linear acceleration approximately every 0.01, s, corresponding to a nominal sampling rate of approximately 100, Hz. The gyroscope measurement is used to estimate angular velocity, while the Euler heading provided by the BNO055 is used as an additional compass-like measurement.

The implementation also uses the measured linear acceleration to maintain a simple distance estimate. The acceleration components are transformed according to the current bearing and passed to the **DistanceTracker**. It is worth noting that under the current implementation, the distance estimate is very unreliable and error skyrockets over time. This estimate should not be used as is unless further improved. This estimated distance is displayed alongside the heading estimate in the realtime visualisation.

For each update of the main Bayesian ring-attractor loop, the accumulated gyroscope measurements are averaged as not to loose measurements if the model run rate is lower than than the aforementioned frequency. The change in the BNO055 heading is also calculated using a circular difference,

$$\Delta\theta = [(\theta_{\text{previous}} - \theta_{\text{current}} + \pi) \bmod 2\pi] - \pi, \quad (5)$$

which prevents discontinuities when the heading crosses the  $0/2\pi$  boundary. The resulting angular velocity and compass change are subsequently supplied to the sensor-fusion system in **SoundBayesianRing**.

### 3.4 360 Camera

The robot is equipped with an omnidirectional camera system that provides a panoramic view around the robot. Rather than processing the raw circular camera image directly, the software transforms the image into a rectangular panoramic representation before visual processing.

The camera is accessed through OpenCV using a standard video device, with camera index 0 corresponding to the primary camera device. The **UnwarpCamera** class implements the preprocessing pipeline. A captured frame is first cropped to the region containing the

circular mirror image and can optionally be undistorted using a previously supplied camera calibration. The circular image is then transformed into a panoramic image using a polar coordinate mapping.

The panoramic frames are subsequently used by the visual-compass component to estimate rotational optical flow. This provides an independent estimate of angular motion that can be combined with the gyroscope measurement in the Bayesian ring attractor.

### 3.5 Microphone Array

An additional sensing modality is provided by a ReSpeaker microphone array. The array is connected to the Raspberry Pi as an audio input device and is detected by searching the available audio devices for a device whose name contains “ReSpeaker”.

The current implementation acquires five audio channels. A block size of 128 samples is used with low-latency input streaming. The microphone-array processing itself uses four microphone channels from the ReSpeaker device, with the remaining channels corresponding to the array’s summed signal and speaker output.

The four microphone signals are arranged as a uniform circular array. The implementation uses an array radius of approximately

$$r = 0.04625, \text{ m.} \tag{6}$$

The direction of arrival (DOA) is estimated using a higher-order delay-and-sum beamforming method. The beamformer evaluates directions from  $0^\circ$  to  $360^\circ$  in  $1^\circ$  increments over the frequency range from 400, Hz to 4200, Hz. The resulting spatial response forms the acoustic observation supplied to the Bayesian ring attractor.

The acoustic observation is converted into a circular probability representation. The maximum of the spatial response determines the estimated direction  $\mu_z$ , while the difference between high- and low-level portions of the response is used to determine the concentration parameter  $\kappa_z$  (details of which will be discussed in further sections). Thus, the microphone array provides both a directional observation and an estimate of its reliability.

## 4 Software Architecture

The software is organized as a modular realtime pipeline in which sensor acquisition, sensor processing, state estimation, and robot control are handled by separate components. The main entry point for the integrated system is `SoundBayesianRing.py`, which coordinates the different modules and provides the realtime processing loop. The architecture is designed so that individual components can be developed and tested independently while still allowing them to operate together on the Raspberry Pi.



## 4.1 Main Realtime Pipeline

The main processing pipeline is implemented in `SoundBayesianRing.py`. At startup, the program initializes the camera, BNO055 IMU, microphone-array input, visual compass, and selected state-estimation model. The estimation model can be configured to use either the Bayesian ring attractor or the circular Kalman filter through the `MODE` parameter. The Bayesian ring attractor is selected using `MODE = "RNN"`, while `MODE = "CKF"` selects the circular Kalman filter.

The loop first requests the most recent camera frame, along with the timestamp and the number of frames missed since last call ( $\#fm$ ). This is because the model runs at a fixed  $dt$  which is calibrated to the frame rate. Therefore if a frame is missed, the angular velocity will be doubled to artificially give the same integrated angular difference as if  $dt$  was doubled, keeping the model on a constant timestep. The main loop obtains the most recent measurements from the different sensing modules. The IMU provides an angular velocity estimate and a change in heading. The visual compass also estimates change in heading. In parallel, the microphone array produces an acoustic spatial response when an audio observation is available. These measurements are assembled into an array vector of angular-motion observations,

$$\omega_t = [\omega_t^{\text{IMU}} \times \#fm \quad \Delta\theta_t^{\text{IMU}}/\Delta t \quad \Delta\theta_t^{\text{visual}}/\Delta t]. \quad (7)$$

The resulting measurements are clipped to a predefined range of  $\pm 8\text{rad/s}$  to avoid extreme jumps.

Next the Direction of arrival distribution is decoded into a  $(\mu, k_z)$  pair using the `sound_to_von_mises` function. For  $\mu$ , it computes the argmax of the distribution before converting it to an angle in rad from  $[-\pi, \pi)$ . As for certainty, it first converts the array into decibels, then thresholds the values to 45 decibels such that low intensity sounds are filtered out as noise. It is then inferred using the difference between the  $s$ 'th lowest data point and  $s$ 'th highest data point,  $\Delta I_s$  (to reduce jumps effects), then scaled using:

$$\kappa_z = (h\Delta I_s)^2 \quad (8)$$

The Bayesian ring attractor receives the processed angular measurements through its `step()` method together with the external input from sound when present. If not present the model runs only on angular velocity measurements.

## 4.2 Data Acquisition and Concurrency

The realtime system uses multiple threads so that high-frequency sensor acquisition does not block the main estimation loop. In particular, the BNO055 is read by a dedicated thread in `IMUReader.py`. The IMU thread samples the sensor approximately every 0.01 s and accumulates the angular velocity measurements until they are requested by the main processing loop. A lock is used to protect the shared measurements while they are being accessed by the estimator.

Camera acquisition is similarly separated from processing when realtime synchronization is enabled. The **Recorder** class continuously captures frames in a dedicated thread and stores the most recent frame together with its capture timestamp. It also records the number of frames captured since the previous frame was retrieved by the main loop. This allows the estimator to account for situations in which the processing loop does not consume every captured frame. Importantly, timestamps are recorded when the camera frame is captured rather than when it is subsequently processed.

Audio acquisition is handled using a low-latency `sounddevice` input stream. The system samples at 16 kHz using five input channels and a block size of 128 samples. The audio callback places incoming data into the audio-processing pipeline, where it is subsequently converted into a spatial direction estimate.

The separation between acquisition and processing is particularly important because the sensors operate at different effective rates. The IMU is sampled at a substantially higher rate than the camera-processing loop, while the audio system operates using short blocks of samples. The main estimator therefore works with the latest available information from each sensor rather than requiring all sensors to produce a measurement at exactly the same instant.

### 4.3 Dynamic Sensor Weighting

Each sensor measuring angular velocity  $i$  carries a trust weight  $k_{v,i}$  which is compared relative to all other sensor weights:

$$\omega_{\text{fused}} = \frac{\sum_i k_{v,i} \omega_i}{\sum_i k_{v,i}} \quad (9)$$

Different sensors may have different accuracies based on the environment such as visual odometry deteriorating in low light situations or IMU compass in magnetic environments. We want an online, unsupervised mechanism that continuously reallocates trust between sensors based on how well they currently agree with one another, without any ground-truth reference signal.

This is modeled by a Hebbian-style rule: sensors that consistently “fire together” should be reinforced together, at the expense of a sensor that disagrees with the group.

We fix the total certainty weight to  $k_{vt}$ :

$$k_{v,1} + k_{v,2} + k_{v,3} = k_{vt} \quad (10)$$

For sensors  $i, j$  with readings  $\omega_i, \omega_j$ , we define the agreement score:

$$a_{ij} = \exp\left(-\frac{(\omega_i - \omega_j)^2}{2\sigma^2}\right) \in (0, 1] \quad (11)$$

This is a Gaussian kernel applied to the reading gap. It measures the similarity as a number between 0 and 1, 1 meaning identical and  $\rightarrow 0$  meaning complete disagreement.

The parameter  $\sigma$  in rad/s sets the tolerance for agreement:

| Gap $ \omega_i - \omega_j $ | Agreement score $a_{ij}$     |
|-----------------------------|------------------------------|
| 0                           | 1.00                         |
| $1\sigma$                   | $\approx 0.61$               |
| $2\sigma$                   | $\approx 0.135$              |
| $3\sigma$                   | $\approx 0.011$ (negligible) |

Each sensor combines the score for each pair by splitting their weight in half and subtracting the opposing pair into  $s_i$ .

$$s_1 = \frac{1}{2}a_{12} + \frac{1}{2}a_{13} - a_{23} \quad (12)$$

$$s_2 = \frac{1}{2}a_{12} - a_{13} + \frac{1}{2}a_{23} \quad (13)$$

$$s_3 = -a_{12} + \frac{1}{2}a_{13} + \frac{1}{2}a_{23} \quad (14)$$

The logic per sensor: it is rewarded (positive contribution) whenever it agrees with either of the other two, and it is penalized (negative contribution) whenever the other two agree strongly with each other while excluding it.

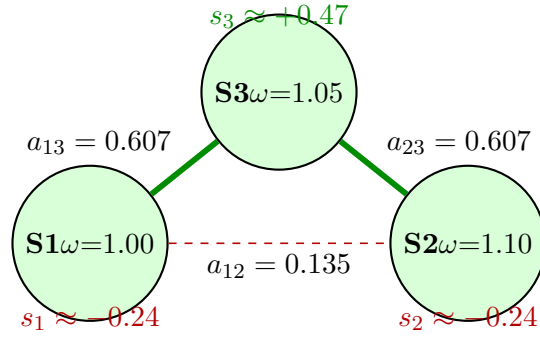


Figure 3: Sensor agreement graph with  $\sigma = 0.05$ , where S3 lies between S1 and S2 in value. S3 agrees moderately with both neighbours ( $a_{13} = a_{23} = 0.607$ ), while the two outer sensors disagree more with each other ( $a_{12} = 0.135$ , dashed edge). Because S3 partially agrees with both, it is reinforced ( $s_3 \approx +0.47$ ) at the expense of the two outer sensors ( $s_1 = s_2 \approx -0.24$ ), even though neither S1 nor S2 is a clear outlier on its own, the “middle” sensor is rewarded simply for bridging the other two.

Given the signal  $s_i$ , the weight is updated multiplicatively:

$$k_{v,i} \leftarrow k_{v,i} \cdot \exp(\eta s_i), \quad i = 1, 2, 3 \quad (15)$$

followed by a renormalization step to restore the exact total:

$$k_{v,i} \leftarrow k_{v,i} \cdot \frac{k_{vt}}{\sum_j k_{v,j}} \quad (16)$$

Here  $\eta > 0$  is the learning rate, it controls the maximum fractional change per step,  $\exp(\pm\eta)$ .

## Recoverability floor

If a sensor’s weight decays very close to zero, its absolute rate of recovery (even under a strongly positive  $s_i$ ) becomes very slow, since  $k_{v,i} \exp(\eta s_i) \approx 0$  for  $k_{v,i} \approx 0$ . To keep recovery possible after a sensor comes back online, a small floor is applied before each update:

$$k_{v,i} \leftarrow \max(k_{v,i}, \varepsilon \cdot k_{vt}), \quad \varepsilon \ll 1 \quad (17)$$

## 4.4 Robot Control Interface

The state-estimation software communicates with the robot-control module through the estimated heading and concentration. The control interface uses a single-element queue so that only the most recent  $(\mu, \kappa)$  pair is retained. If a new estimate arrives while an older estimate is waiting to be processed, the older value is discarded. This prevents a backlog of outdated commands from accumulating during realtime operation.

The robot-control thread converts the estimated heading into a motion command. The sign and magnitude of  $\mu$  determine whether the robot should turn left, turn right, or move forward. The concentration  $\kappa$  controls the duration of the resulting action, allowing the controller to modify its behaviour according to the certainty of the Bayesian estimate. The resulting commands are then passed to the low-level motor interface implemented using the PCA9685 PWM controller.

## 4.5 Logging and Realtime Visualization

The main pipeline records the state-estimation output and sensor measurements to a CSV file. For realtime-synchronized operation, each entry contains the camera capture timestamp, estimated mean direction, estimated concentration, the three angular-motion measurements, and the current values of their associated certainty weights. This provides a complete record of the estimator state and its sensor inputs for subsequent analysis.

The system also provides a lightweight Flask web interface for realtime visualization. The processed camera frames are encoded as JPEG images and streamed as an MJPEG feed to <http://<pi-ip>:5000/>. The visualization can display the current heading estimate, its concentration, and the estimated distance, allowing the behavior of the estimator to be monitored while the robot is operating.

## 5 Bayesian Ring attractor

This section outlines the Bayesian ring attractor implementation and some details about the parameters available.

### 5.1 Initialisation and Network Parameters

Constructor:

```
def __init__(self, N, dt, tau, kappa_phi, k_v, k_z,
             w_const, w_quad, kappa_0, phi_0, stoch_corr):
```

$N$  is the number of neurons in the ring. Each neuron  $i$  has a preferred head direction  $\phi_i$ , distributed evenly around the circle:

```
self.phi = np.linspace(-np.pi, np.pi, N, endpoint=False)
```

The initial population activity stored in activity history array  $r$ , is set as a cosine bump centred on  $\phi_0$  with amplitude  $\kappa_0$ ,

$$r_t^{(i)} = \kappa_t \cos(\phi_i - \mu_t) : \quad (18)$$

```
self.r.append(kappa_0 * np.cos(self.phi - phi_0))
```

#### 5.1.1 Weight Matrix Construction

The recurrent weight matrix decomposes into three components:

$$W(v_t) = W^{\text{const}} + W^{\text{sym}} + W^{\text{asym}}(v_t) \quad (19)$$

However, every sensor has its own  $W^{\text{asym}}(v_t)$  matrix to be scaled by its respective angular velocity.

Each is precomputed in `__init__`:

```
diff = self.phi[:, None] - self.phi[None, :]
self.W_sym = w_sym * (2/N) * np.cos(diff)
for i in range(len(self.k_v)):
    self.W_asym.append((2 / N) * np.sin(self.diff) * w_asym[i])
self.W_const = (w_const / N) * np.ones((N, N))
```

The symmetric cosine component  $W_{ij}^{\text{sym}} = \frac{2}{N} w_{\text{sym}} \cos(\phi_i - \phi_j)$  stabilises the bump position. The antisymmetric sine component  $W_{ij}^{\text{asym}} = \frac{2}{N} w_{\text{asym}} \sin(\phi_i - \phi_j)$  rotates the bump in proportion to signed angular velocity. The constant component adds background excitation.

For the model to behave as an attractor network the following has to hold:

$$w_{\text{sym}} = \frac{1}{\tau} + \frac{1}{\kappa_{\phi} + \kappa_v}, \quad w_{\text{asym}} = \frac{\kappa_v}{\kappa_{\phi} + \kappa_{vt}} \quad (20)$$

```
w_sym = 1/tau + 1/(kappa_phi + k_v)
w_asym = [k_v / (kappa_phi + k_v_total) for k_v in self.k_v]
```

### 5.1.2 External Input

The external cosine input in the code is not scaled simply by  $k_z dt$ . Instead, the code finds a parameter  $\alpha$  satisfying the following equation where  $I$  is the Bessel function.

$$\alpha \cdot A(\alpha) = \kappa_z dt, \quad A(\alpha) = \frac{I_1(\alpha)}{I_0(\alpha)} \quad (21)$$

This is solved numerically using the following function:

```
def xi_fun_inv(self, dt):
    f = lambda alpha: alpha * self.A_Bessel(alpha) - dt
    sol = root_scalar(f, bracket=[0.001, 50], method='brentq')
    return sol.root

self.I_ext = self.xi_fun_inv(k_z * dt)
```

`I_ext` is precomputed once on initialization, but is recomputed whenever  $\kappa_z$  is passed as input to the `step()` function.

## 5.2 Network Dynamics: `step()`

Each call to `step()` advances the population activity by one Euler step.

First, all the matrices for the asymmetrical components are computed as the weights of the sensors change dynamically by default. The Matrix  $W$  is implemented according to equation 19 :

```
W = self.W_sym + self.W_const
for i in range(len(dy)):
    W += self.W_asym[i] * dy[i]
```

This incorporates angular velocity from the angular displacement input  $dy$ .

The full dynamics are represented by the following equation:

$$r_{t+dt} = r_t - \frac{1}{\tau} r_t dt - \sigma_{\text{corr}} r_t dt + W(v_t) r_t dt - w_{\text{quad}} \left( \frac{\pi}{N} \mathbf{11}^T f(r_t) \right) r_t dt + I_{\text{ext}} \cos(\phi - z_t) \quad (22)$$

implemented as:

```

M = (np.pi / self.N) * np.ones([self.N, self.N])

self.r.append((self.r[-1]
    - self.stoch_corr * self.r[-1] * self.dt # stochastic
    correction
    - 1 / self.tau * self.r[-1] * self.dt # decay
    + np.dot(W, self.r[-1]) * self.dt # angular velocity
    integration, recurrent stabilization
    - self.w_quad * np.dot(M, f_act(self.r[-1])) * self.r[-1] *
    self.dt # quadratic inhibition
    + z_cancel * self.I_ext * np.cos(self.phi - z))) # absolute
    heading info (external input)

```

The matrix  $M = \frac{\pi}{N}\mathbf{1}\mathbf{1}^T$  is used to compute the total rectified population activity  $\frac{\pi}{N} \sum_i [r^{(i)}]_+$ . This quadratic inhibition term creates stable bump fixed point:  $\kappa^* = \frac{w_{\text{sym}} - 1/\tau}{w_{\text{quad}}}$ .

Note that the external input term has no  $dt$  factor as the  $dt$  scaling computed in `xi_fun_inv` via the argument  $\kappa_z dt$ .

The HD enter the model in radians through  $z$  and angular velocity in rad/s through  $dy$

### 5.3 Decoding $\mu_t$ and $\kappa_t$

The mean direction and certainty are decoded from the population activity at each step:

```

basis = np.array([np.cos(self.phi), np.sin(self.phi)]) # (2, N)
theta = (2/self.N) * (basis @ self.r[-1]) # (2,)
mu = np.arctan2(theta[1], theta[0])
kappa = np.linalg.norm(theta)

```

This recovers the Cartesian belief vector  $\boldsymbol{\theta} = (\kappa_t \cos \mu_t, \kappa_t \sin \mu_t)$  via the projection:

$$\theta_1 = \frac{2}{N} \sum_i r^{(i)} \cos \phi_i = \kappa_t \cos \mu_t, \quad \theta_2 = \frac{2}{N} \sum_i r^{(i)} \sin \phi_i = \kappa_t \sin \mu_t \quad (23)$$

This holds because the  $\cos$  and  $\sin$  basis functions are evenly-spaced around the circle so:  $\frac{2}{N} \sum_i \cos^2 \phi_i = 1$  and  $\frac{2}{N} \sum_i \sin \phi_i \cos \phi_i = 0$ . The mean  $\mu_t$  is then the angle of  $\boldsymbol{\theta}$  and the certainty  $\kappa_t$  is its magnitude.

## 5.4 Parameters and Their Effects

### 5.4.1 $N$ : Number of Neurons

$N$  controls the resolution of the ring. Lowering  $N$  does not significantly impact performance. It was kept at  $N = 30$

### 5.4.2 tau : Membrane Time Constant

$\tau$  sets the timescale of the passive leak decay  $-\frac{1}{\tau}r_t$ . Under the balance condition  $w_{\text{sym}} = \frac{1}{\tau} + \frac{1}{\kappa_\phi + \kappa_v}$ , the  $1/\tau$  from the leak is exactly cancelled by the  $1/\tau$  in the symmetric weight. Consequently  $\tau$  disappears from the dynamics and is a free parameter that does not affect the output. However it should satisfy  $dt \ll \tau$  for numerical stability.

### 5.4.3 kappa\_phi : Inverse Diffusion Constant

$\kappa_\phi$  encodes how stable the true head direction process is. A larger  $\kappa_\phi$  means less diffusion so the HD changes slowly and the network should weight its current estimate highly relative to new input. It appears in both  $w_{\text{sym}}$  and  $w_{\text{asym}}$  with similar effect. For example,  $w_{\text{asym}} = \kappa_v / (\kappa_\phi + \kappa_v)$ , so increasing  $\kappa_\phi$  reduces the weight given to angular velocity observations. However, this means that for true dead reckoning it should be set to 0 otherwise it will add a constant reductive bias to change in angle.

### 5.4.4 k\_vt : Total Angular Velocity Reliability

$\kappa_{vt}$  encodes how reliable the angular velocity signal is. It appears in  $w_{\text{asym}} = \kappa_v / (\kappa_\phi + \kappa_{vt})$ : a larger  $\kappa_v$  compared to  $\kappa_\phi$  increases the asymmetric weight, causing the bump to rotate faster in response to velocity input. It also controls speed of certainty decay.

### 5.4.5 kappa\_z : HD Observation Reliability

$\kappa_z$  encodes how reliable the angular velocity signal is. A larger  $\kappa_z$  shifts  $\mu_t$  faster toward the observed direction and grows  $\kappa_t$  more quickly when the observation is confirmatory. The input strength is the concentration  $\alpha$  satisfying  $\alpha \cdot A(\alpha) = \kappa_z dt$  as discussed previously.

### 5.4.6 w\_quad : Quadratic Inhibition Strength

$w_{\text{quad}}$  controls the global inhibition that stabilises bump amplitude. The fixed point certainty is  $\kappa^* = \frac{w_{\text{sym}} - 1/\tau}{w_{\text{quad}}} = \frac{1/(\kappa_\phi + \kappa_v)}{w_{\text{quad}}}$ . A larger  $w_{\text{quad}}$  lowers  $\kappa^*$  so the bump stabilizes at a lower value.

## 5.5 Output

The relevant values representing the heading belief can be accessed via the `mu` and `kappa` fields. Both are arrays containing the full history of the belief with the latest value at index [-1]. Similarly, the activity history can be accessed in field `r`.

## 6 Results

### 6.1 Model performance score

We introduce here a metric to capture model performance taking into account certainty and trial difficulty. The idea is to take circular error:  $\epsilon_i = \left| \left( (\hat{\theta}_i - \theta_i + \pi) \bmod 2\pi \right) - \pi \right|$  and



weigh it by the associated certainty at each time step. We thus take the weighted average over the trial, weighing by kappa:  $E_{\kappa} = \frac{\sum_{i=1}^T \kappa_i \cdot \epsilon_i}{\sum_{i=1}^T \kappa_i}$ .

We also need to quantify trial difficulty as a trial with many turns in a short time will induce more error than a stationary trial, as seen in our phase analysis shown later. This is computed as the total angular displacement over the trial length.  $D = \frac{\sum_{i=1}^{T-1} |\theta_{i+1} - \theta_i|}{T}$

We can also introduce the constant  $\alpha$  to regulate how much the difficulty score contributes. We then combine such that a higher value indicates a good model:

$$\text{WCAS} = \frac{1}{E_{\kappa}} + \alpha \cdot D \quad (24)$$

We define this score as Weighted Certainty Average Score (WCAS), we measure ignoring the first 7 seconds of the trial.

## 6.2 h optimization

We look at the effect of h (external input sensitivity) on the WCAS defined. We drive the robot and collect angular velocity and sound measurements, while also collecting ground truth. We found that h could be optimized with a value of 1.16 in this specific trial.

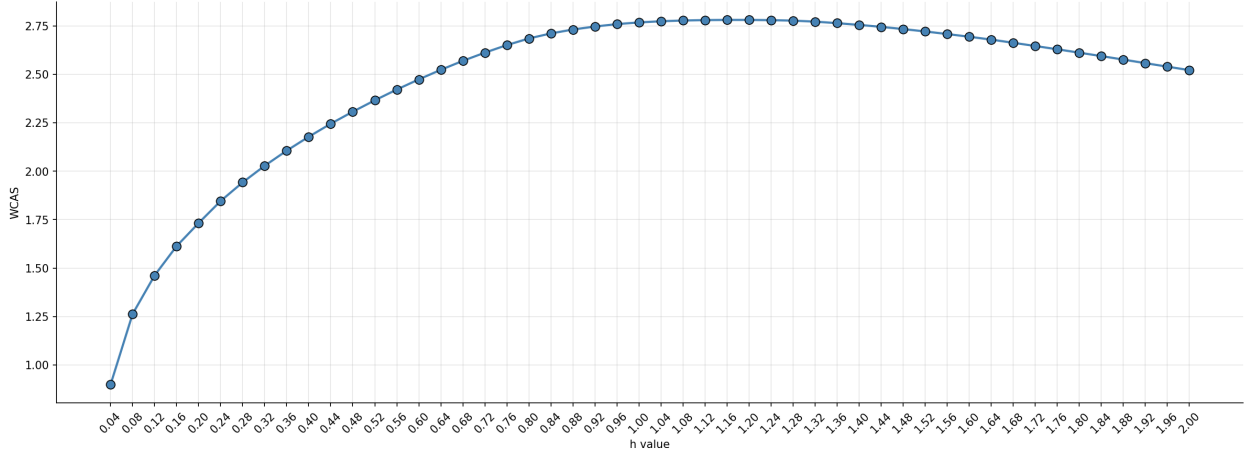


Figure 4: external input sensitivity h vs WCAS

## 6.3 Effect of stationary vs motion phases

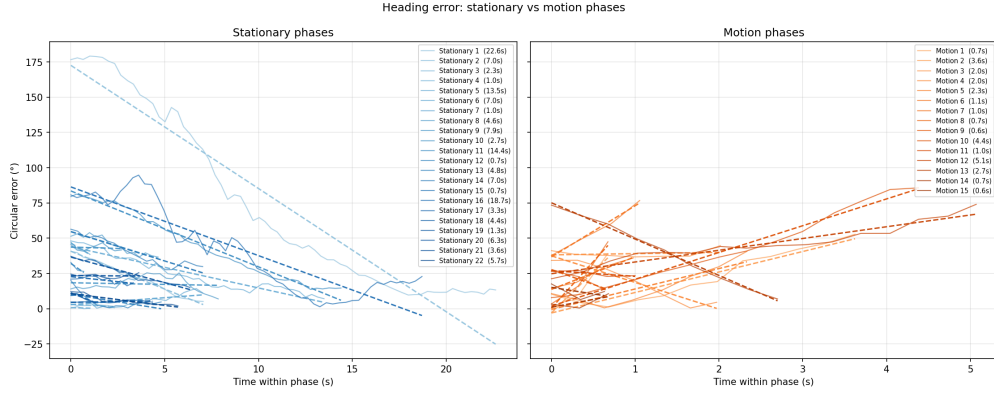


Figure 5: Trial for  $h=0.2$  non adaptive. Stationary phases is a period of time over .5 seconds where the angular velocity stayed under 0.15 rad/s during that time. A motion phase is equivalent except velocity must stay above 0.15 rad/s

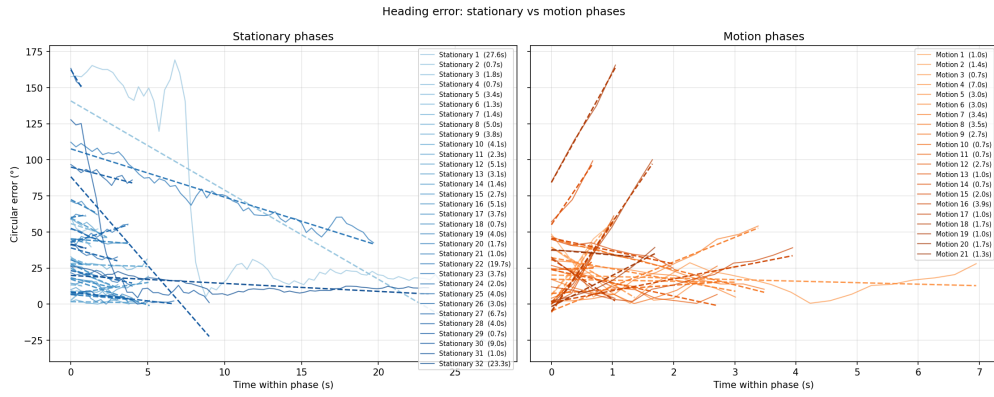


Figure 6: Trial for  $h=0.2$  adaptive. Stationary phases is a period of time over .5 seconds where the angular velocity stayed under 0.15 rad/s during that time. A motion phase is equivalent except velocity must stay above 0.15 rad/s

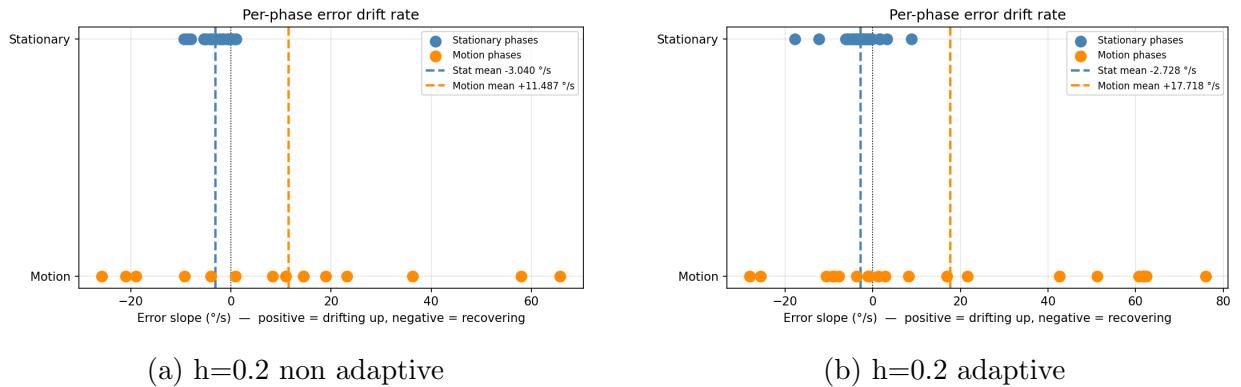


Figure 7: Gradient distribution for stationary phase in blue and motion phase in orange

It seems clear here that stationary phases let the model update itself to correct the error. Motion phases are more varied but are more likely to increase the error. This variability is expected as the robot may turn towards the ground truth, coincidentally reducing error which may cause negative gradients.

This justifies the difficulty in comparing different trials together which WCAS tries to account for.

## 6.4 Adaptive weights difference

Using the same methodology as the  $h$  parameter sweep, we obtain the difference between the adaptive weights and non adaptive weights performance. We see a slight increase in performance with the adaptive weights.

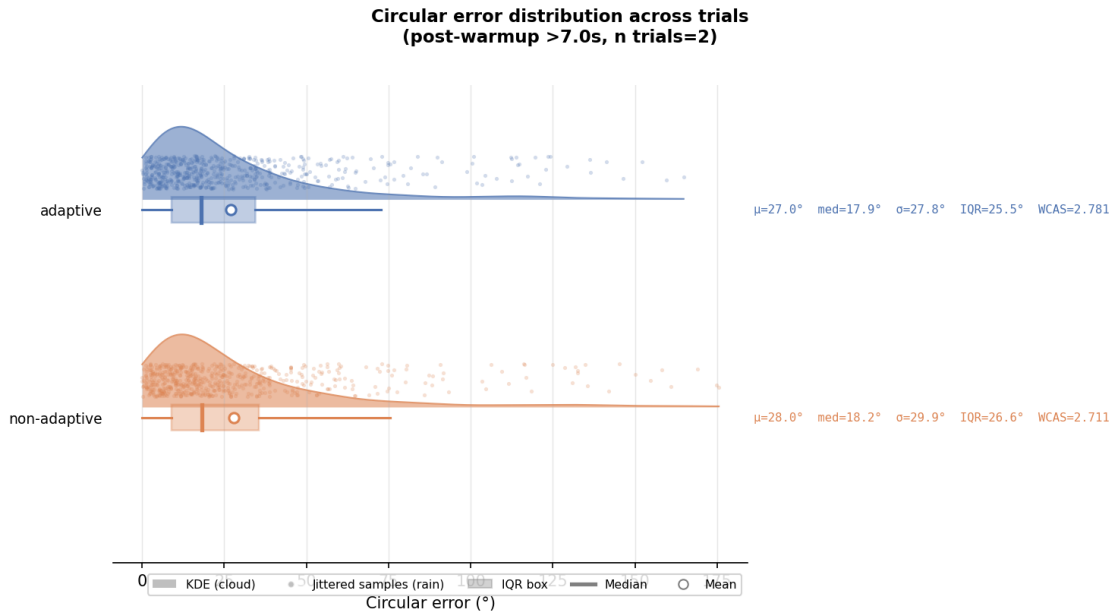


Figure 8: Adaptive weights and non adaptive weights method ran on the same data

## 6.5 Ground truth comparison

Thus using the optimal  $h$  parameter together with adaptive weights yields the following estimates over time:

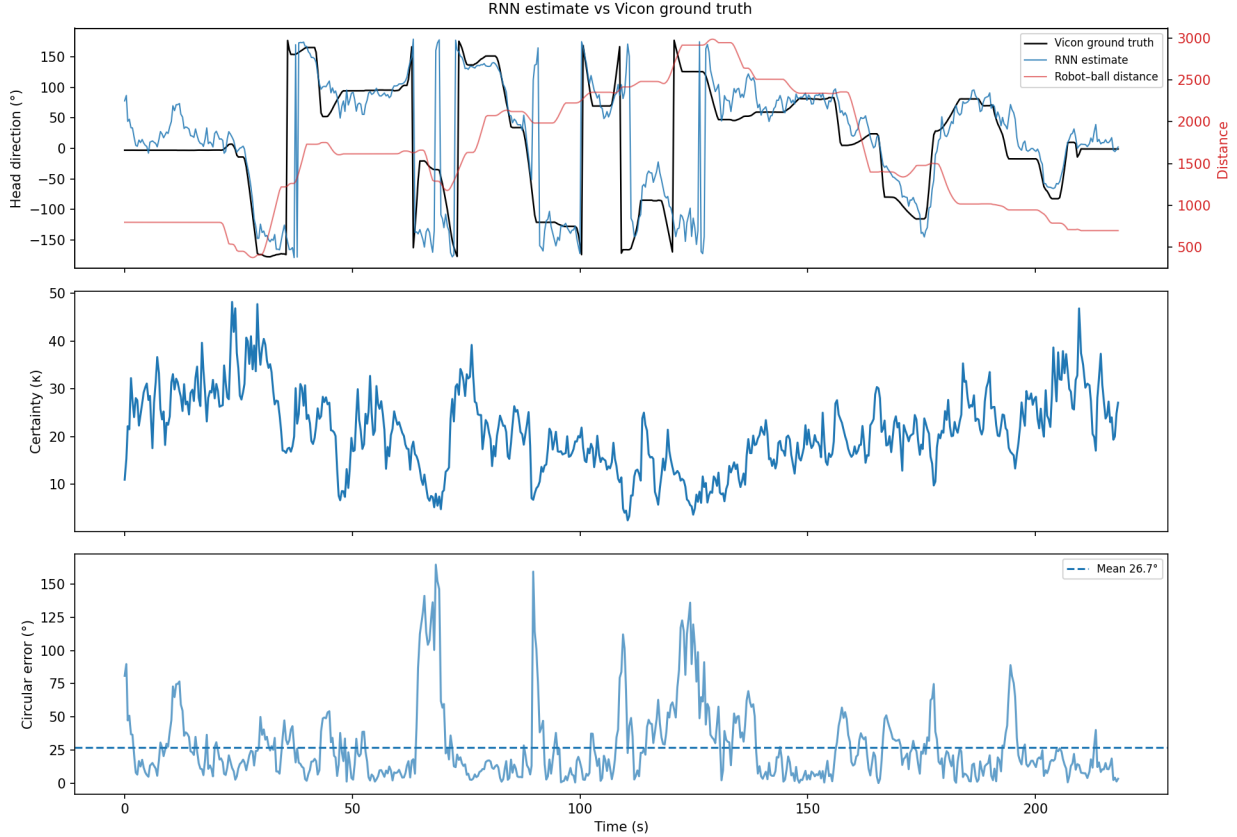


Figure 9: True Head direction over time vs Estimate over time

## 7 Conclusion

This project implemented a Bayesian ring attractor for circular state estimation and evaluated its use in a physical robotic homing system. The system combines multiple sensory modalities, including visual odometry from an omnidirectional camera, angular measurements from a BNO055 inertial measurement unit, and acoustic direction-of-arrival observations from a ReSpeaker microphone array. These heterogeneous observations are processed independently before being integrated within a common circular representation.

A central feature of the system is its probabilistic representation of heading. Rather than producing only a single directional estimate, the Bayesian ring attractor represents both the estimated direction  $\mu$  and its associated certainty  $\kappa$ . This allows observations to be weighted according to their reliability and provides the robot with information about not only where the target is estimated to be, but also how confident the system is in that estimate. The resulting uncertainty can also be incorporated into the robot’s behaviour, allowing the homing controller to respond differently depending on the confidence of the current estimate.

The project therefore demonstrates how the principles of Bayesian inference in ring attractor networks can be transferred from a theoretical neural model to a physical robotic system. The combination of circular probabilistic estimation, multimodal sensory integration, and

uncertainty-aware control provides a framework for robust heading estimation in situations where individual sensors may be noisy, incomplete, or temporarily unreliable. More generally, the physical platform provides a basis for investigating how Bayesian continuous attractor models can be used for multimodal navigation and spatial estimation in increasingly complex environments.

## References

- [1] Melanie A. Basnak et al. “Multimodal cue integration and learning in a neural representation of head direction”. In: *Nature Neuroscience* 28.8 (2025), pp. 1729–1740. DOI: 10.1038/s41593-024-01823-z.
- [2] Anna Kutschireiter et al. “Bayesian inference in ring attractor networks”. In: *Proceedings of the National Academy of Sciences* 120.9 (2023), e2210622120. DOI: 10.1073/pnas.2210622120.